

Chapitre 3 : Bash avancé

TD/TP

Table des matières

1	Travaux Dirigés	1
1.1	Structures de contrôle.....	1
1.1.1	Analyse.....	1
1.1.2	Construction.....	2
1.2	Paramètres spéciaux.....	3
1.2.1	Paramètres d'appel	3
1.2.2	Statut d'exécution	4
1.3	Commandes utiles.....	4
1.3.1	Commande test	4
1.3.2	Commande read	5
2	Travaux Pratiques	5
2.1	factorielle.sh.....	5
2.2	factorielle_recuratif.sh	6
2.3	afficher_nom.sh.....	6
2.4	afficher_contenu.sh	6
2.5	mon_seq.sh.....	7
2.6	calculatrice.sh	7
2.7	mon_cat.sh	7
2.8	tester_script.sh.....	7

1 Travaux Dirigés

Les exercices suivants approfondissent quelques-unes des notions vues en cours.

1.1 Structures de contrôle

1.1.1 Analyse

Indiquez ce que font et affichent les programmes shell suivants, et nommez les constructions qu'ils utilisent.

1.1.1.1 Programme

```
n=10
m=7
echo "$n + $m = $((n + m))"
```

1.1.1.2 Programme

```
n=10
if ((n % 2 == 0)) then
    echo $n est pair
else
    echo $n est impair
fi
```

1.1.1.3 Programme

```
n=5
while ((n >= 0)) do
    echo $n
    ((n--))
done
```

1.1.1.4 Programme

```
for n in $(seq 5); do
    echo $((n * 2))
done
```

1.1.1.5 Programme

```
for log in $(ls /var/log/*.log); do
    wc -l $log
done
```

1.1.2 Construction

Ecrivez en shell les programmes suivants.

1.1.2.1 Départ

Affiche la séquence suivante :

```
5
4
3
2
1
Partez !
```

1.1.2.2 Somme

Variable de départ : N, contenant un entier strictement positif.

Affiche les N premiers entiers, en partant de 1, suivis de leur somme.

1.1.2.3 Concatenation

Variables de départ : chaîne, contenant une chaîne de caractères quelconque, et répétition, contenant un entier supérieur ou égal à 0.

Concatène chaîne à elle-même répétitions fois, en ajoutant un espace à chaque fois. Si répétition est égal à 0, chaîne est laissée à sa valeur initiale. Exemple : "salut !" répétée 2 fois donne "salut ! salut ! salut !".

1.1.2.4 Triangle

Variable de départ : hauteur, contenant un entier strictement positif.

Affiche un triangle plein de hauteur hauteur, représenté à l'aide des caractères X.

Exemple de triangle de hauteur 4 :

```
X
XX
XXX
XXXX
```

1.1.2.5 Executables

Variable de départ : repertoire, contenant le chemin d'accès absolu à un répertoire.

Affiche le chemin d'accès absolu aux fichiers exécutables de repertoire.

Note : vous devez vous aider du man de la commande test.

1.2 Paramètres spéciaux

Vous pouvez réaliser les exercices suivants dans le shell bash.

1.2.1 Paramètres d'appel

Rappel : dans un script, \$# désigne le nombre de paramètres du script, \$0 désigne le chemin d'accès par lequel le script a été lancé, \$i, $i \geq 1$, désigne le paramètre numéro i et \$* désigne la totalité des paramètres pour $i \geq 1$.

Développez le script parametres.sh qui affiche, sur des lignes distinctes, le nombre de ses paramètres, ainsi que les paramètres 0 à 3 inclus.

Appelez votre script plusieurs fois avec un nombre d'arguments allant de 0 à 5. Appelez-le également en utilisant un chemin d'accès tantôt relatif, tantôt absolu, pour le désigner.

Observez l'affichage produit : quelle valeur a \$3 quand il n'y a que deux paramètres ? Les paramètres de rang supérieur à 3 sont-ils affichés ? Que se passe-t-il quand vous appelez le script avec les arguments Hello World ! ? Comment faites-vous pour que la phrase Hello World ! constitue un seul et même argument ?

Votre script ne sait afficher que ses trois premiers paramètres. Développez-en une seconde version, parametres2.sh, qui affiche \$0 ainsi que tous les paramètres suivants, quel que soit leur nombre, un par ligne, en utilisant une boucle for.

La commande interne `shift` décale les paramètres d'un cran vers la gauche. Après un appel à `shift`, `$1` vaut `$2`, `$2` vaut `$3`, `$3` vaut `$4`, etc. et le nombre de paramètres `$#` est décrémenté. Développez une troisième version de votre script, `arguments3.sh`, qui produit le même affichage que `parametres2.sh`, mais en s'appuyant sur une boucle `while` et en n'utilisant que le paramètre `$1`. Cette construction vous sera utile dans le TP : elle permet de facilement retrouver les valeurs associées à options dont on ne connaît pas la position.

1.2.2 Statut d'exécution

Rappel : `bash` stocke dans le paramètre spécial `$?` le statut d'exécution de la dernière commande qu'il a exécutée.

Exécutez la commande `cat /etc/passwd` et relevez son statut d'exécution. Recommencez avec la commande `cat /` et interprétez les résultats que vous avez obtenu. Faites des essais avec d'autres commandes, `ls`, `touch`, `mkdir`, etc. et vérifiez qu'elles se conforment bien à la convention donnée dans le cours.

La commande interne `exit` permet de terminer le script en cours d'exécution. Si un paramètre numérique est donné à `exit`, il constitue le statut d'exécution du script.

Pour la mettre en pratique, développez le script `statut.sh`. Quand il est appelé avec un seul paramètre, le script affiche ce paramètre sur sa sortie standard et se termine avec le statut d'exécution « succès ». Quand le nombre de paramètres est différent de 1, le script affiche le message d'erreur « usage : status.sh PARAMETRE » sur sa sortie erreur et se termine avec le statut d'exécution « échec ».

1.3 Commandes utiles

Voici deux commandes internes utiles pour le développement des scripts du TP.

1.3.1 Commande test

La commande `test` permet de comparer des valeurs chaînes ou entiers, comme on le fait dans un programme classique écrit en Python ou en C. Elle permet de plus de tester l'existence de fichiers, répertoires ou liens. Elle permet également de combiner plusieurs tests avec les opérateurs booléens standard. La commande `test` s'abrège `[ ... ]` pour plus de clarté (noter l'espace obligatoire après le `[` et avant le `]`).

Voici quelques exemples d'appel :

```
$ test "bonjour" = "salut"
$ echo $?
1
$ [ "salut" = "salut" ]
$ echo $?
0
$ [ -f /etc/passwd ]
$ echo $?
0
$
```

Lisez attentivement le man de la commande pour en découvrir toutes les possibilités.

1.3.2 Commande read

La commande read lit une ligne de l'entrée standard et la stocke dans la variable spécifiée en argument. Voici des exemples d'appel :

```
$ read ligne
Salut, ca va ?
$ echo "${ligne}"
Salut, ca va ?
$ read ligne < /etc/passwd
$ echo "${ligne}"
$ echo "${ligne}"
root:x:0:0:/root:/usr/bin/zsh
$
```

2 Travaux Pratiques

Il s'agit de développer les scripts spécifiés dans les paragraphes suivants, en respectant les conventions vues en cours :

- Les messages normaux sont écrits sur la sortie standard (canal 1), et les messages d'erreur sur la sortie erreur (canal 2).
- Le statut d'exécution des scripts est 0 en cas de succès, et un nombre positif en cas d'erreur.

2.1 factorielle.sh

Synopsis

```
factorielle.sh ENTIER_NATUREL
```

Description

Affiche la factorielle de l'entier naturel passé en argument. Le calcul est fait en mode itératif, i.e. par une boucle.

Exemples

```
$ ./factorielle.sh
usage : factorielle.sh ENTIER_NATUREL
$ ./factorielle.sh 0
1
$ ./factorielle.sh 6
720
$
```

2.2 factorielle_recuratif.sh

Mêmes synopsis, description et exemples que pour `factorielle.sh`. Le calcul est fait en mode récursif, i.e. le script se rappelle lui-même.

2.3 afficher_nom.sh

Synopsis

```
afficher_nom.sh [OPTIONS]
```

Description

Affiche le chemin d'accès de l'entrée spécifié par les options. Les options peuvent être saisies dans un ordre quelconque. Elles peuvent être répétées, auquel cas c'est la dernière valeur donnée qui est prise en compte.

```
-r REPERTOIRE
    spécifie le répertoire de l'entrée, défaut : "" (chaîne vide)
-n NOM
    spécifie le nom de l'entrée, défaut : "main"
-e EXTENSION
    spécifie l'extension de l'entrée, défaut : ".c"
```

Exemples

```
$ ./afficher_nom.sh
main.c
$ ./afficher_nom.sh -e '' -r /tmp -n foo
/tmp/foo
$ ./afficher_nom.sh -r /var -e log
/var/main.log
$ ./afficher_nom.sh -d /var
afficher_nom.sh : option -d inconnue
$
```

2.4 afficher_contenu.sh

Synopsis

```
afficher_contenu.sh [ENTREE]...
```

Description

Affiche le nom de chaque entrée spécifiée, suivi de son contenu. Si l'argument est un répertoire, son contenu est donné par `ls`. Si l'argument est un fichier, son contenu est donné par `cat`. Si l'argument n'est ni un répertoire ni un fichier, le script affiche 'pas une entree'.

Exemples

```
$ ./afficher_contenu.sh
$ ./afficher_contenu.sh ~ foo factorielle.sh
```

```
/home/lamia :  
Desktop Documents Downloads Music Pictures Videos  
foo :  
pas une entree  
factorielle.sh :  
# !/usr/bin/env bash  
...  
$
```

2.5 mon_seq.sh

Ré-implémente à l'identique la commande `seq`, sans ses options et en ne considérant que des valeurs entières. Voir le manuel pour le synopsis et la description de la commande.

Contrainte : le script n'utilise bien sûr pas la commande `seq`. Les entiers sont générés grâce aux développements arithmétiques.

2.6 calculatrice.sh

Synopsis

```
calculatrice.sh
```

Description

Calcule les expressions arithmétiques entières entrées interactivement par l'utilisateur. La commande affiche l'invite "> " pour attendre une expression. La commande se termine quand l'utilisateur saisit "exit".

Exemples

```
$ ./calculatrice.sh  
> 1 + 2  
3  
> 1 + 2 * 3  
7  
> exit  
$
```

2.7 mon_cat.sh

Ré-implémente à l'identique la commande `cat`, avec ses options `-n`, `-s` et `-E`. Voir le manuel pour le synopsis et la description de la commande.

Contrainte : le script n'utilise bien sûr pas la commande `cat`. Il lit les fichiers à l'aide de la commande interne `read`.

2.8 tester_script.sh

Synopsis

```
tester_script.sh SCRIPT TEST...
```

Description

Teste un script dans différents cas de test. La commande traite des scripts prenant exactement un argument sans espaces et affichant exactement un résultat sans espaces.

Chaque TEST est spécifié sous la forme ARGUMENT=RESULTAT. La commande exécute SCRIPT de cette façon : `$ SCRIPT ARGUMENT`.

Statut d'exécution

Le statut d'exécution est 0 si tous les tests réussissent, 1 si au moins un test échoue, 2 si un argument est incorrect, 3 si la syntaxe d'appel est incorrecte.

Exemples

```
$ ./tester_script.sh foo
usage : tester_script.sh SCRIPT TEST...
$ ./tester_script.sh foo 1=1
tester_script.sh : foo : pas un script executable
$ ./tester_script.sh ./factorielle.sh 0=1 1=1 3:6 4=24
OK 0 => 1
OK 1 => 1
tester_script.sh : 3:6 : test mal specifie
NOK 4 => 24, obtenu : 12
3 tests, 66% OK
$ ./tester_script.sh ./factorielle.sh 0=1 1=1 3=6 4=24
OK 0 => 1
OK 1 => 1
OK 3 => 6
NOK 4 => 24, obtenu : 12
4 tests, 75% OK
$ # après correction du script factorielle.sh :
$ ./tester_script.sh ./factorielle.sh 0=1 1=1 3=6 4=24
OK 0 => 1
OK 1 => 1
OK 3 => 6
OK 4 => 24
4 tests, 100% OK
$
```